



Dwight Look College of
ENGINEERING
TEXAS A&M UNIVERSITY

Team 45: LES Parts Database UI

Bi-Weekly Update 2

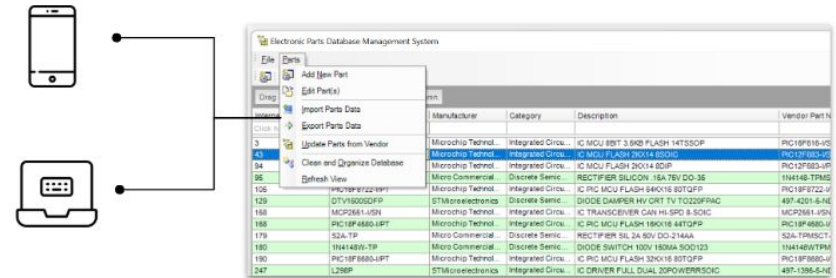
Anthony Beneventi, Jamaal Bloom, Jaswin Jabbal

Sponsor: John Lusher

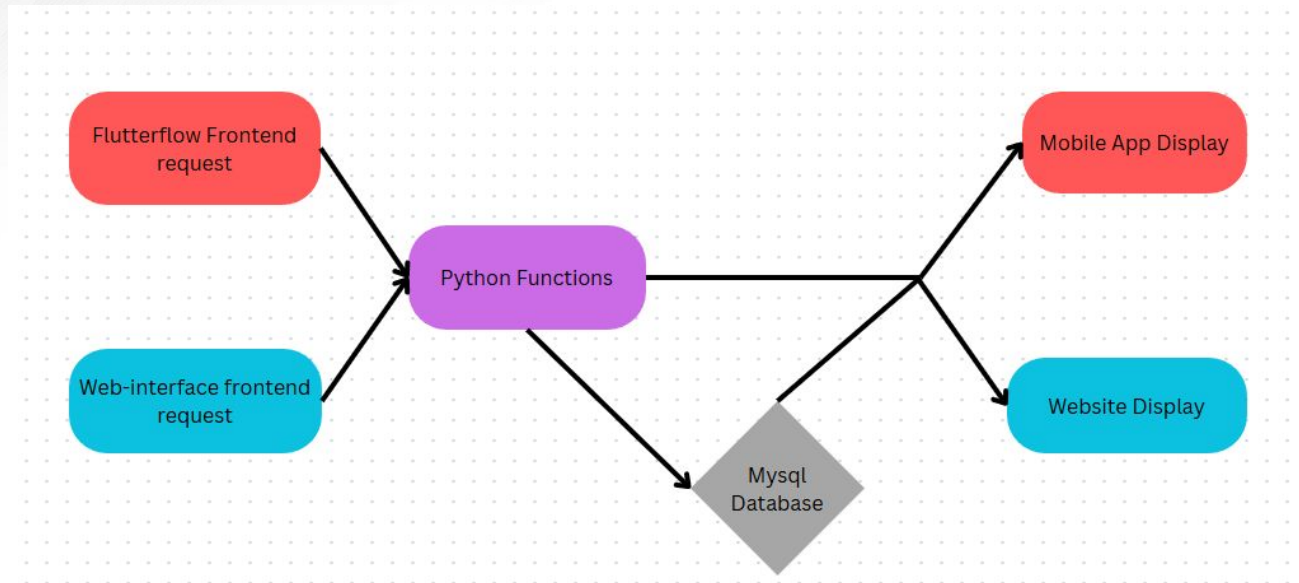
TA: Sabyasachi Gupta

Project Summary

- The current database requires manually updating parts and limits multiple users working at the same time.
- Develop a web and mobile application that will allow for the auto population and filtering of part information

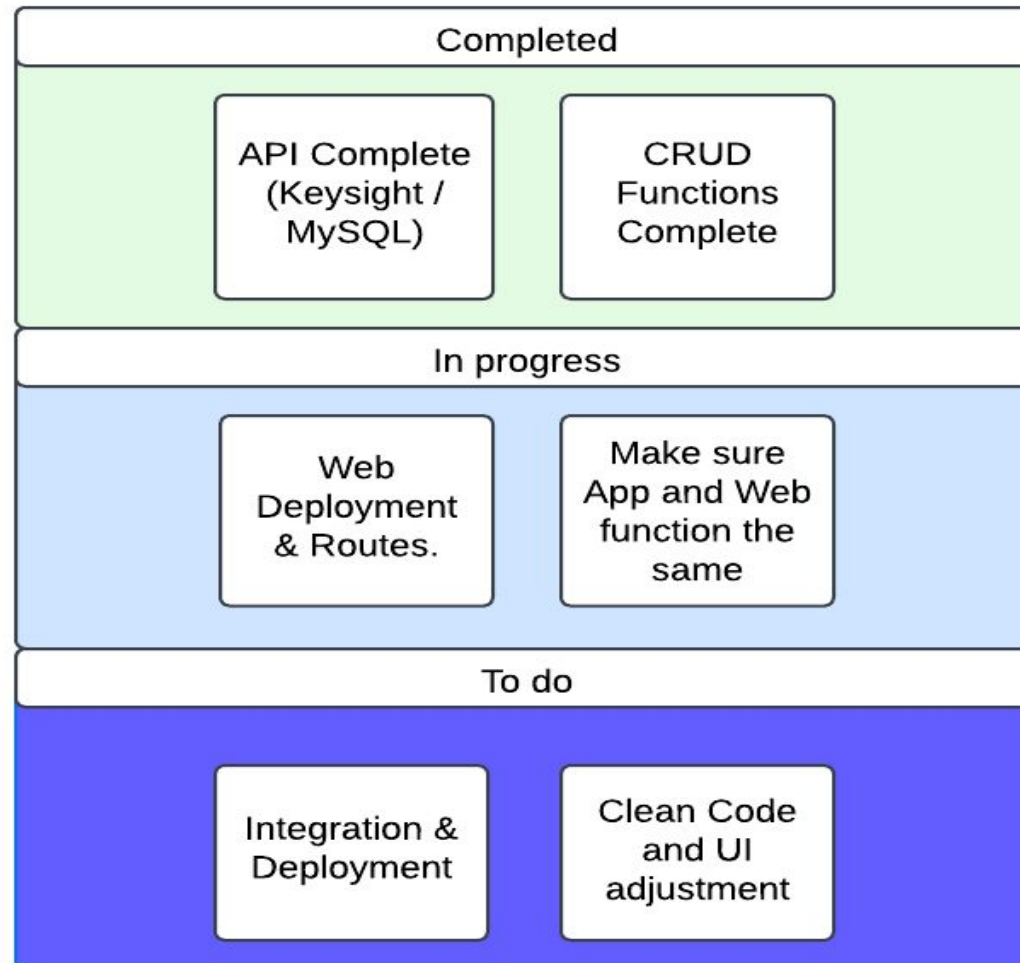


Project/Subsystem Overview



Team: 
 Jamaal: 
 Anthony: 
 Jaswin: 

Project Timeline



Web Application UI

Jaswin Jabbal

Accomplishments since Update 1 6+ hrs of effort	Ongoing progress/problems and plans until the next presentation
- Wrote new models based on MySQL schema - Creating routes for CRUD operations - debugging	- Route functionality/ validation - Frontend data consistency - Deployment Environment

[illegible]

Exploring issuing of database creation within the app



Python Backend

Jamaal Bloom

Accomplishments since Update 1 12 hrs of effort	Ongoing progress/problems and plans until the next presentation
• Converted code to callable API. • Tested on Flutterflow	• Get and connect to an online database • Format output for mobile and web use.





IOS Application

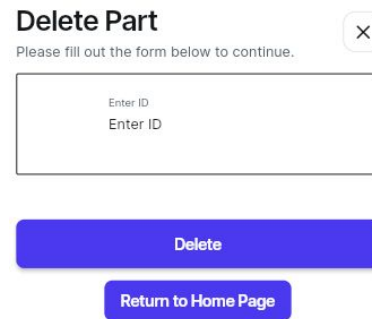
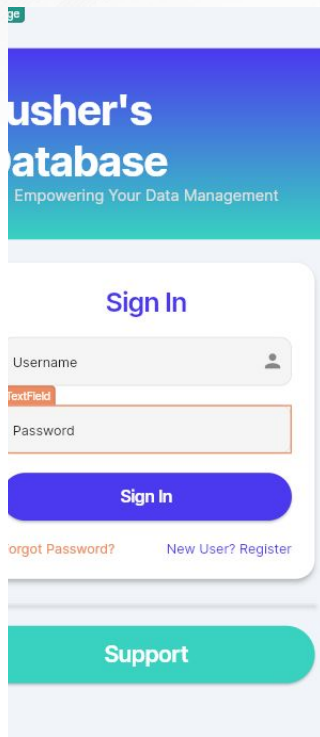
Anthony Beneventi

Accomplishments since Update 1 8+ hrs of effort	Ongoing progress/problems and plans until the next presentation
• User Authority Feature • Multiple User Functionality	• incorporate kesight API from WEB subsystem into IOS App. • Integration into online database

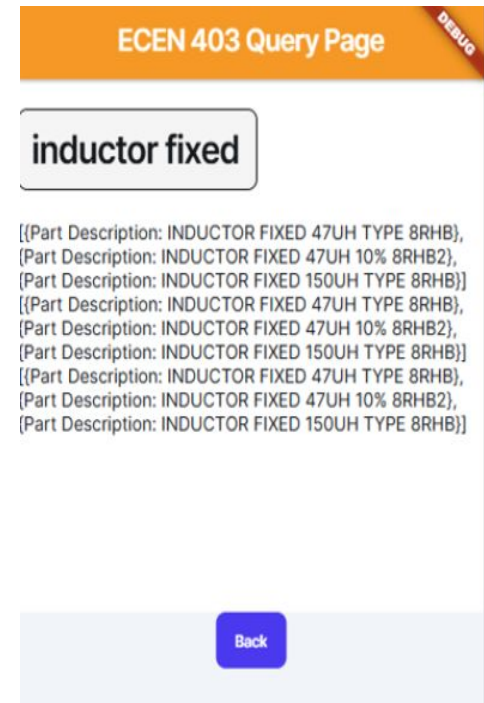


IOS Application

Anthony Beneventi



Delete function



Example of search query, "Inductor fixed."

Functioning Forgot password +
New User registration.

Execution Plan

Task Assignments	Owner	1/13/25	1/20/25	1/27/25	2/3/25	2/10/25	2/17/25	2/24/25	3/3/25	3/10/25	3/17/25	3/24/25	3/31/25	4/7/25	4/14/25	4/21/25	4/28/25	Due Date		Key	
Finish Subsystems	Team																			Planned	
Convert code to callable API	Jamaal																	1/27/25		In Progress	
New models based on DB requirement	Jaswin																			Complete	
Fix Update/Insert Butt	Anthony																			Behind Schedule	
Connect with online database	Jamaal																	2/10/25			
Routes based on new models	Jaswin																				
Integrate API	Anthony																				
Format frontend and backend requests	Jamaal																	2/24/25			
Data Consistency	Jaswin																				
Consolidate App	Anthony																				
Finish integration	Team																				
Implement failsafes	Jamaal																	3/17/25			
Deployment	Jaswin																				
System validation	Anthony																				
Finialize/Polish code	Jamaal																	3/31/25			
Deployment	Jaswin																				
Optimize code	Anthony																				
Final Presentation	Team																	4/14/25			
Showcase/ Demo	Team																	4/24/25			



Validation Plan

Paragaph #	Test Name	Success Criteria	Methodology	Status	Responsible Engineer(s)
3.2.1.4	Database Sync	The web and mobile applications uses the same database and changes visible from both	Attempt changes from multiple devices and ensure consistency	Untested	Jamaal
3.2.1.1	API Correctness	The API pulls correct information from the supplier	Compare recieved information with suppliers website	TESTED	Jamaal
3.2.6.2	Request Correctness	The backend correctly recieves and handles the frontend requests	Send frontend request from both applications and examine results	Untested	Jamaal
3.2.1.3	Multiple Requests	Application can handle multiple requests at the same time	Send multiple requests	Untested	Jamaal
3.2.2.1	Model Validation	All DB models align correctly with the updated schema used by Jamaal	Compare Flask SQLAlchemy models against the latest database schema	Untested	Jaswin
3.2.4.1	Route Functionality	Form submission routes to the correct display without errors	Manually fill out and submir forms in the application, verifying data correctness	Untested	Jaswin
3.2.4.2	Frontend Data Consistency	Submitted data is correctly displayed on a corresponding UI template	Manual validation by cross-checking displayed records with DB entries	Untested	Jaswin
3.2.6.1	Deployment Environment	All required dependencies are correctly installed and configured on the end machine	Validate with Docker, verify installation logs, ensure WGSi server	Untested	
3.2.1.2	CRUD Operation Test	Pulling apart information from the MySQL Database	Access a local MySQL Server with dummy datta	Untested	Anthony
3.2.1.4	Updated Database	The API Correctly updates the entire database	Checking previous and updated dababases to ensure completion	Untested	Anthony
3.2.1.5	Request Correctness	Having all requested components, such as User authority	Giving specific permission to individual User admins	Untested	Anthony
3.2.1.3	Application Deployed	Uploaded on the cloud and ready for use	Able to be sent for beta testing	Untested	Anthony



Dwight Look College of

ENGINEERING
TEXAS A&M UNIVERSITY

Team 45: Anthony Beneventi, Jamaal Bloom, and Jaswin Jabbal

Questions?